

Fast Volume Rendering with Spatiotemporal Reservoir Resampling (Supplemental Document)

DAQI LIN, University of Utah, USA

CHRIS WYMAN, NVIDIA, USA

CEM YUKSEL, University of Utah, USA

ACM Reference Format:

Daqi Lin, Chris Wyman, and Cem Yuksel. 2021. Fast Volume Rendering with Spatiotemporal Reservoir Resampling (Supplemental Document). *ACM Trans. Graph.* 40, 6, Article 279 (December 2021), 4 pages. <https://doi.org/10.1145/3478513.3480499>

1 RIS FOR PATH INTEGRALS

This subsection provides a more rigorous derivation of the RIS estimator of the path integral proposed in Section 3.3 in the paper. We assume the path tracing process being a random walk that generates n paths λ^i ($i \in \{1, \dots, n\}$) that differ by lengths (or emission/scatter type in the volumetric case), responsible for n non-overlapping parts ($\int_{\lambda^i} F(\lambda^i) d\lambda^i$) of the path integral $L = \int_{\Lambda} F(\lambda) d\lambda$ (where λ can be any length or type). In other words, $L = \sum_{i=1}^{\infty} \int_{\Lambda^i} F(\lambda^i) d\lambda^i$. An ordinary 1-sample Monte Carlo estimator simply sums the contributions from all sampled paths, i.e. $\langle L \rangle_{MC} = \sum_{i=1}^n \frac{F(\lambda^i)}{p(\lambda^i)}$. Note that n is a random variable and can pick any value from 0 to ∞ . We know that $\langle L \rangle_{MC}$ is an unbiased estimator of the path integral L . However, we want to take advantage of RIS to avoid evaluating F for all paths. The traditional form of RIS assumes that all candidate samples are taken from the same sampling domain to estimate the same integral. Now, we want to use RIS to estimate a sum of integrals using candidate samples taken from the sampling domain of each integral, and only evaluate F for the chosen sample λ^r . This requires a slightly different definition of RIS (the "1/ M " term is removed from the estimator). We now show that the RIS estimator

$$\langle L \rangle_{ris} = \frac{F(\lambda^r)}{\hat{p}(\lambda^r)} \sum_{i=1}^n \frac{\hat{p}(\lambda^i)}{p(\lambda^i)}$$

where λ^r is chosen from $\lambda^1, \dots, \lambda^n$ according to the weight $w(\lambda^i) = \frac{\hat{p}(\lambda^i)}{p(\lambda^i)}$ is also an unbiased estimator of the path integral, i.e. $\mathbb{E}[\langle L \rangle_{ris}] = \mathbb{E}[\langle L \rangle_{MC}]$. The proof goes as follows,

$$\mathbb{E}[\langle L \rangle_{ris}] = \mathbb{E} \left[\mathbb{E} \left[\frac{F(\lambda^r)}{\hat{p}(\lambda^r)} | \lambda^1, \dots, \lambda^n \right] \cdot \sum_{i=1}^n w(\lambda^i) \right]$$

Authors' addresses: Daqi Lin, University of Utah, USA; Chris Wyman, NVIDIA, USA; Cem Yuksel, University of Utah, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2021 Copyright held by the owner/author(s). Publication rights licensed to ACM. 0730-0301/2021/12-ART279 \$15.00

<https://doi.org/10.1145/3478513.3480499>

$$\begin{aligned} &= \mathbb{E} \left[\sum_{j=1}^n \left(\frac{F(\lambda^j)}{\hat{p}(\lambda^j)} \cdot \frac{w(\lambda^j)}{\sum_{i=1}^n w(\lambda^i)} \right) \cdot \sum_{i=1}^n w(\lambda^i) \right] \\ &= \mathbb{E} \left[\sum_{j=1}^n \frac{F(\lambda^j)}{\hat{p}(\lambda^j)} \cdot \frac{\hat{p}(\lambda^j)}{p(\lambda^j)} \right] \\ &= \mathbb{E} \left[\sum_{j=1}^n \frac{F(\lambda^j)}{p(\lambda^j)} \right] = \mathbb{E}[\langle L \rangle_{MC}]. \quad \square \end{aligned}$$

2 COMPARISON WITH VERTEX REUSE

Section 4.2 in the paper notes we can either resample paths by reusing path vertices $\mathbf{x}_i$ or by reusing directions ω_i . We chose to reuse directions; while costs are somewhat higher for direction reuse, it reduces noise fairly significantly. This is because paths act a little like virtual point lights under reuse, which leads to more singularities and fireflies. However, if these could be reduced, it may make sense to switch back to vertex reuse for the improved performance. See Figure 1 to compare visually between vertex and directional reuse.

3 DENOISED RESULT

In Figure 2, we compare denoising applied to both our baseline and our resampling technique; both using the new OptiX 7.3 temporal denoising mode. While OptiX produces amazingly denoised results in both cases, the better sampling provided by our technique preserves much higher frequency details in the animation, while the baseline gives a smoother, more washed out look. Part of this is also due to the improved motion vectors we provide with our novel temporal reprojection plus velocity resampling.

Please see the supplementary video to compare the denoised results under animation.

4 PSEUDOCODE

We provide the pseudocode of our fast volume rendering with spatiotemporal reservoir resampling (Volumetric ReSTIR) in this supplemental document. Algorithm 1 gives an outline of the full algorithm, ignoring surface bounces for simplicity. Algorithm 2 shows the reservoir class we used for storing and streaming the samples. Algorithm 3, Algorithm 5, and Algorithm 6 shows four individual stages of our algorithm: initial sampling, temporal reuse, spatial reuse, and final shading. We provide the detail of how we resample the target function $\hat{p}$ in Algorithm 4.

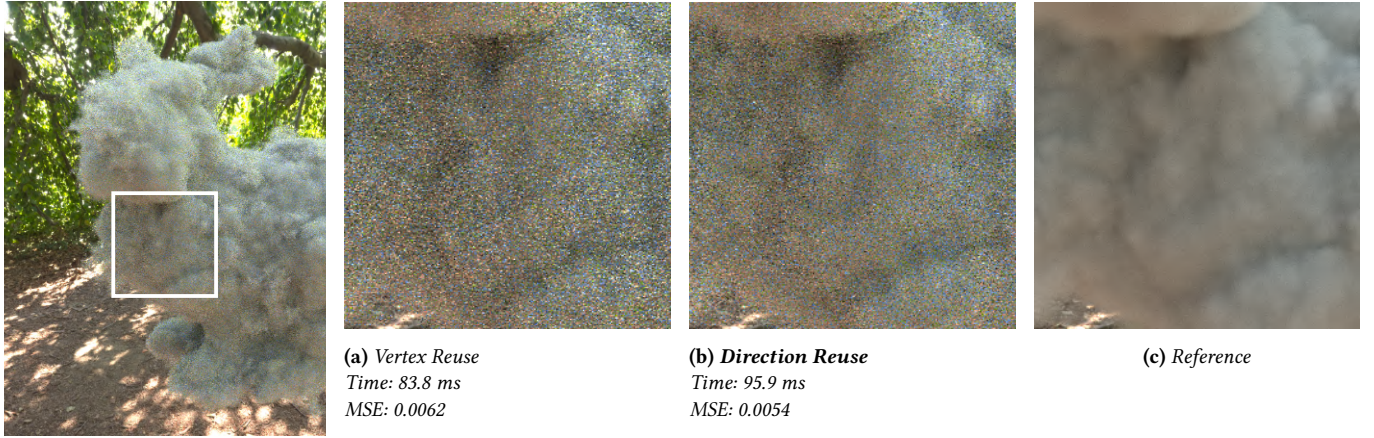


Fig. 1. Compare vertex and direction reuse. (We reuse directions.) Here we show 7-bounce multiple scattering in the Bunny Cloud scene. While vertex reuse provides a slight performance win, it introduces additional sampling variance due to large weights of potentially irrelevant paths during spatial reuse.



Fig. 2. The OptiX 7.3 temporal denoiser applied to both our baseline and our method, using (top) single and (bottom) 7-bounce multiple scattering.

Algorithm 1: Full algorithm.

```

1 function VolumetricReSTIR()
2   allocate an image size array of reservoirs
3   foreach  $q \in \text{Image}$  do
4     | reservoirs[ $q$ ]  $\leftarrow$  InitialSampling( $q$ )
5   end
6   foreach  $q \in \text{Image}$  do
7     | reservoirs[ $q$ ]  $\leftarrow$  TemporalReuse( $q$ )
8   end
9   foreach  $q \in \text{Image}$  do
10    | reservoirs[ $q$ ]  $\leftarrow$  SpatialReuse( $q$ )
11  end
12  prevFrameReservoirs  $\leftarrow$  reservoirs
13  foreach  $q \in \text{Image}$  do
14    | pixel color  $\leftarrow$  FinalShading( $q$ )
15  end

```

Algorithm 2: Pseudocode of the Reservoir class.

```

1 class Reservoir
2    $\lambda \leftarrow \text{null}$ 
3   // the content of the path sample can be seen in line 3, Algorithm 4.
4    $w_{\text{sum}} \leftarrow 0$ 
5    $M \leftarrow 0$ 
6    $\hat{p} \leftarrow 0$ 
7   function update( $p_i, \hat{p}_i, \lambda_i$ )
8     | if null sample then
9       |  $M \leftarrow M + 1$ 
10    else
11      |  $w_i \leftarrow \hat{p}_i / p_i$ 
12      |  $w_{\text{sum}} \leftarrow w_{\text{sum}} + w_i$ 
13      |  $M \leftarrow M + 1$ 
14      | // Generate a random number  $\xi$ .
15      | if  $\xi < w_i / w_{\text{sum}}$  then
16        | |  $\lambda \leftarrow \lambda_i$ 
17        | |  $\hat{p} \leftarrow \hat{p}_i$ 
18  function combineReservoir( $\hat{p}_i, r_i, m$ )
19    |  $w_i \leftarrow \hat{p}_i / (r_i \cdot \hat{p}) \cdot r_i \cdot w_{\text{sum}} \cdot m$ 
20    |  $w_{\text{sum}} \leftarrow w_{\text{sum}} + w_i$ 
21    |  $M \leftarrow M + r_i \cdot M$ 
22    | if  $\xi < w_i / w_{\text{sum}}$  then
23      | |  $\lambda \leftarrow \lambda_i$ 
24      | |  $\hat{p} \leftarrow \hat{p}_i$ 

```

Algorithm 3: Pseudocode of the initial sampling pass.

```

1 function InitialSampling( $q$ )
2   Reservoir  $r$ 
3   [ $\mathbf{x}_0, \omega_0$ ]  $\leftarrow$  ray origin and direction from pixel  $q$ 
4   //  $M$  is the number of random walks
5   for  $m \leftarrow 1$  to  $M$  do
6     Reservoir  $r_m$ 
7      $\hat{p} \leftarrow 1$ 
8      $p \leftarrow 1$ 
9      $\lambda_0 \leftarrow$  Path with only camera vertex
10    for  $i \leftarrow 1$  to  $K$  do
11      [ $z_i, \sigma_i^*(\mathbf{x}_i), T^*$ ]  $\leftarrow$  RegularTracking( $\mathbf{x}_{i-1}, \omega_{i-1}$ )
12       $\lambda_i \leftarrow$  AddPathVertex( $\lambda_{i-1}, \mathbf{x}_i$ )
13      if  $z_i == z_s$  then
14        | if  $i == 1$  then
15          | |  $p \leftarrow p \cdot T^*$ 
16          | |  $\hat{p} \leftarrow \hat{p} \cdot L^s T^*$ 
17          | |  $r_m.\text{update}(p, \hat{p}, \lambda_i)$ 
18        | else
19          | |  $r_m.\text{update}(\text{null sample})$ 
20        | break
21      if  $\sigma_i(\mathbf{x}_i) == 0$  then
22        |  $r_m.\text{update}(\text{null sample})$ 
23        | break
24       $p \leftarrow p \cdot \sigma_i^*(\mathbf{x}_i) T^*$ 
25       $\hat{p} \leftarrow \hat{p} \cdot T^*$ 
26      if volume contains emission then
27        |  $\hat{p}' \leftarrow \hat{p} \cdot \sigma_a(\mathbf{x}_i) L_e^m(\mathbf{x}_i)$ 
28        | // the superscript  $e$  marks an emission path
29        |  $r_m.\text{update}(p, \hat{p}', \lambda_i^e)$ 
30        | // Scattering event. ( $p_{\text{NEE}}$  is the pdf of the light sample)
31        | [ $L_s, \mathbf{x}'_{i+1}, \omega'_i, p_{\text{NEE}}$ ]  $\leftarrow$  SampleLight( $\mathbf{x}_i$ )
32        | // ray marching with a larger step size
33        |  $\tilde{T} \leftarrow$  EstimateVolumetricShadow( $\mathbf{x}_i, \mathbf{x}'_{i+1}$ )
34        |  $\hat{p} \leftarrow \hat{p} \cdot \sigma_s(\mathbf{x}_i) \rho(\mathbf{x}_i, -\omega_{i-1}, \omega'_i) L_s \tilde{T}$ 
35        |  $p \leftarrow p \cdot p_{\text{NEE}}$ 
36        |  $r_m.\text{update}(p, \hat{p}, \text{AddPathVertex}(\lambda_i, \mathbf{x}'_{i+1}))$ 
37        | // Prepare for the next bounce.
38        |  $\omega_i \leftarrow$  SamplePhaseFunction( $\mathbf{x}_i$ )
39        |  $\hat{p} \leftarrow \hat{p} \cdot \sigma_s(\mathbf{x}_i) \rho(\mathbf{x}_i, -\omega_{i-1}, \omega_i)$ 
40        |  $p \leftarrow p \cdot \rho(\mathbf{x}_i, -\omega_{i-1}, \omega_i)$ 
41    end
42    // Set  $M$  to 1 since we are using the path integral RIS.
43     $r_m.M \leftarrow 1$ 
44     $r.\text{combineReservoir}(\hat{p}(r_m.\lambda), r_m, 1)$ 
45  end
46  return  $r$ 

```

Algorithm 4: Resample target function.

```

1 function ResampleTargetFunction( $\lambda, q$ )
2   [ $\mathbf{x}_0, \omega_0$ ]  $\leftarrow$  ray origin and direction from pixel  $q$ 
   //  $k'$  is the number of scattering events (plus 1 for emissive paths1).  $\mathbf{x}_{k'+1}$ 
   and  $\omega_{k'}$  are the position and direction of the light sample.
3   [ $k', isEmission, z_1, \omega_1, z_2, \omega_2, \dots, z_{k'}, \omega_{k'}, \mathbf{x}_{k'+1}$ ]  $\leftarrow \lambda$ 
4    $\hat{p} \leftarrow 1$ 
5   for  $i \leftarrow 1$  to  $k'$  do
6      $\mathbf{x}_i = \mathbf{x}_{i-1} + z_i \omega_{i-1}$  // Direction reuse
7      $\tilde{T} \leftarrow \text{RayMarching}(\mathbf{x}_{i-1}, \mathbf{x}_i)$ 
8     if  $z_i == z_s$  then
9       if  $i == 1$  then
10         $\hat{p} \leftarrow L^s \tilde{T}$ 
11        return  $\hat{p}$ 
12       else
13        return 0
14     if  $isEmission$  then
15       return  $\hat{p} \cdot \sigma_a(\mathbf{x}_i) \tilde{T} L_e^m(\mathbf{x}_i)$ 
16      $\hat{p} \leftarrow \hat{p} \cdot \sigma_s(\mathbf{x}_i) \tilde{T}$ 
17      $\hat{p} \leftarrow \hat{p} \cdot \rho(\mathbf{x}_i, -\omega_{i-1}, \omega_i)$ 
18   end
   // Transmittance of the NEE segment
19    $\tilde{T} \leftarrow \text{RayMarching}(\mathbf{x}_{k'}, \mathbf{x}_{k'+1})$ 
20    $\hat{p} \leftarrow \hat{p} \cdot \tilde{T}$ 
21   return  $\hat{p}$ 

```

¹Directly visible surface emission also counts as an emission path, with the corresponding terms ("if emission path") in Equation 15 and 25 in the paper changed to $L^s(\mathbf{x}_1 \rightarrow \mathbf{x}_0)$.

Algorithm 5: Pseudocode of spatial/temporal reuse.

```

1 function TemporalReuse( $q$ )
2    $q' \leftarrow \text{TemporalReprojection}(q, \text{reservoirs}[q])$ 
3   return CombineReservoirs(reservoirs[ $q$ ],
   prevFrameReservoirs[ $q'$ ],  $q, q'$ )
4 function SpatialReuse( $q$ )
5    $S \leftarrow \text{pickSpatialNeighbors}(q)$ 
6   return CombineReservoirs(reservoirs[ $q$ ], {reservoir[ $q'$ ] |
    $q' \in S$ },  $q, \{q' \in S\}$ )
7 function CombineReservoirs( $r_0, r_1, \dots, r_N, q_0, q_1, \dots, q_N$ )
8   Reservoir  $s$ 
9   foreach  $r_i \in \{r_0, \dots, r_N\}$  do
10     $p_{\text{sum}} \leftarrow 0$ 
11     $k \leftarrow 0$ 
   // Compute MIS weight
12    foreach  $q_s \in \{q_0, \dots, q_N\}$  do
13       $\hat{p}_{q_s}(r_i, \lambda) \leftarrow \text{ResampleTargetFunction}(r_i, \lambda, q_s)$ 
14       $p_{\text{sum}} \leftarrow p_{\text{sum}} + \hat{p}_{q_s}(r_i, \lambda) \cdot r_i.M$ 
15       $k \leftarrow k + r_i.M$ 
16    end
17     $m \leftarrow \frac{\hat{p}_{q_i}(r_i, \lambda)}{p_{\text{sum}}/k}$  // MIS weight
18     $\hat{p}_q(r_i, \lambda) \leftarrow \text{ResampleTargetFunction}(r_i, \lambda, q_0)$ 
19     $s.\text{combineReservoir}(\hat{p}_q(r_i, \lambda), r_i, m)$ 
20  end
21  return  $s$ 

```

Algorithm 6: Final shading.

```

1 function FinalShading( $q$ )
2    $r \leftarrow \text{reservoirs}[q]$ 
   // Analytical transmittance values are computed.
3    $f(r, \lambda) \leftarrow \text{compute integrand}$ 
4   return  $\frac{f(r, \lambda)}{\hat{p}(r, \lambda)} \frac{r \cdot w_{\text{sum}}}{r.M}$ 

```
